AIGC 5500 Midterm, Group 01: Ranjay Salmon, Ankit Yadav, Ripan Deep Kour, Teja Ekkurthi

Which architecture best reads a short request?

Comparing a CNN, an LSTM and a Transformer against a simple baseline

Our problem

When someone types a short request, can we tell what they want, and which kind of model does that best?

We tried to identify how much the architecture matters when every model gets the same data, budget and tuning, and whether a simple model already does the job.

To identify this, we used four models

- **Baseline:** logistic regression on word counts, which ignores word order
- **1D CNN:** looks for local patterns of 2 to 4 words
- **Bi-LSTM:** reads the sentence in both directions, keeping state
- **Transformer:** lets every word look directly at every other word

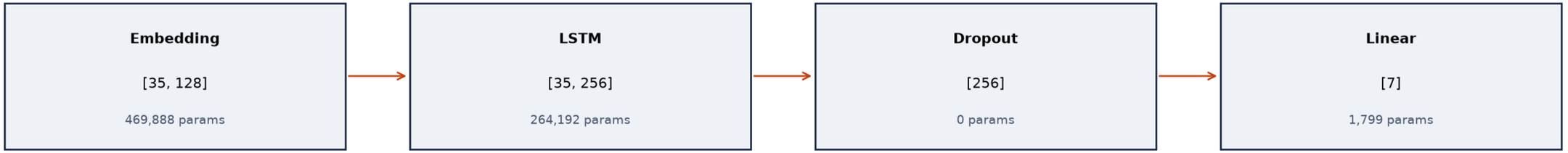
Three seeds each, one component removed per family, then a second dataset.

The three deep models, from our code

TextCNN (the three Conv1d + BatchNorm1d pairs are parallel branches, shown in run order): 588,095 parameters



TextRNN (Bi-LSTM): 735,879 parameters



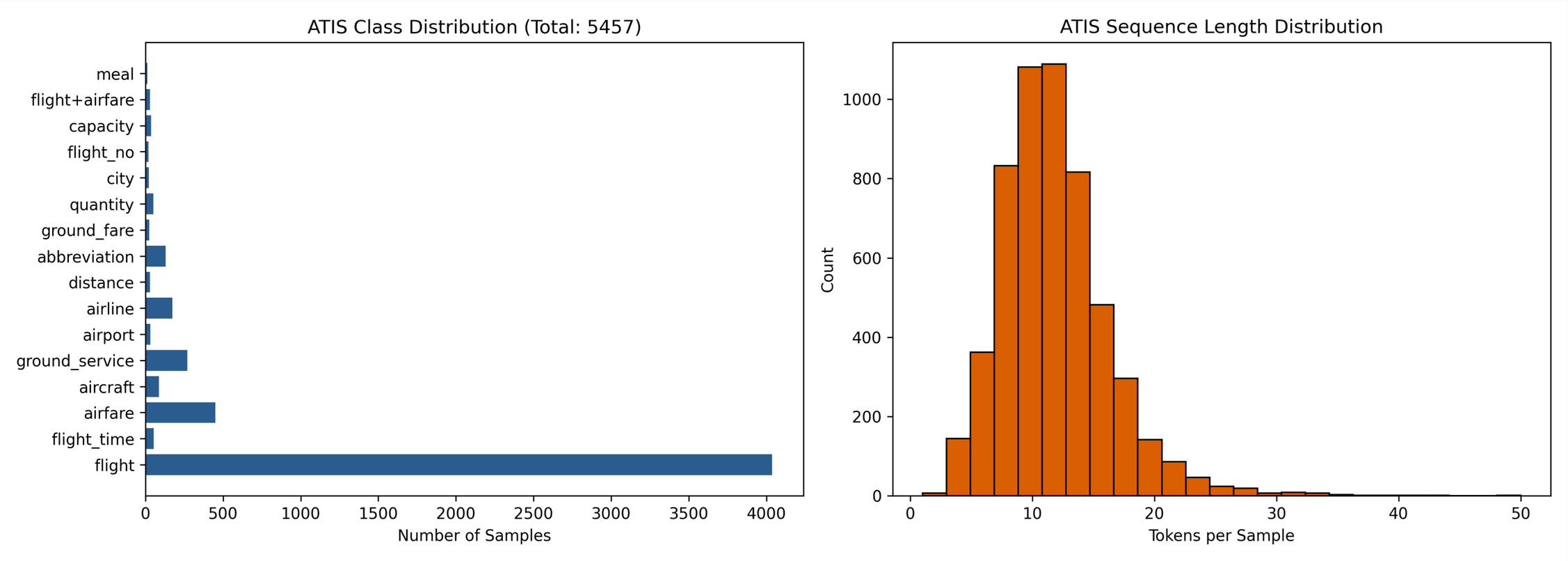
TextTransformer: 735,751 parameters



Output shape excludes batch size 64. Traced from models.py, SNIPS vocabulary 3,671, length 35.

The data contained two kinds of request

SNIPS: voice-assistant commands, 7 balanced intents, 14,262 unique utterances. **ATIS:** flight queries, 16 intents, 5,457 unique utterances, about 74% “flight”.



We ran it through one shared process

- **Split:** stratified 70/15/15
- **Encode:** train-only vocabulary, 35 tokens
- **Train:** AdamW, 20 epochs, three seeds
- **Select:** best epoch on validation
- **Score:** test macro F1, reported for every seed

Macro F1 counts every intent equally, however rare.

Round 1

Results of our first runs

Model	SNIPS mean F1	ATIS mean F1
Baseline (TF-IDF)	0.9837	0.7426
1D CNN	0.9828	0.7953
Bi-LSTM	0.9813	0.8161
Transformer	0.9808	0.8192

Original pipeline, commit 9a647c8. Each seed also used a different data split.

The trends we saw first

- **The simple model kept up on SNIPS:** all four sat near 0.98, with gaps smaller than seed variation
- **The Transformer looked best on ATIS,** but one seed reversed the deep models' lead over the baseline
- **Removing components changed almost nothing:** all three moved F1 by less than the seed spread

What we found, and what we updated

What we found	What we updated
2% of SNIPS and 8–9% of ATIS test rows also in training	Removed duplicates before splitting
Only the baseline was tuned	Same validation grid for every family
Baseline used different inputs and training	Same token inputs, optimizer and budget
Each seed changed the data split	One fixed split per dataset
Deep confusion matrices came from untrained models	Built from trained models’ predictions
Peak memory was not measured	Measured for all models in fresh processes

Round 2

The rerun exposed a new problem

Model	SNIPS mean F1	ATIS mean F1
Baseline	0.9764	0.1831
1D CNN	0.9791	0.8132
Bi-LSTM	0.9783	0.8037
Transformer	0.9759	0.7718

The baseline wasn’t bad at ATIS; it had underfit. So we widened the learning-rate grid for every family.

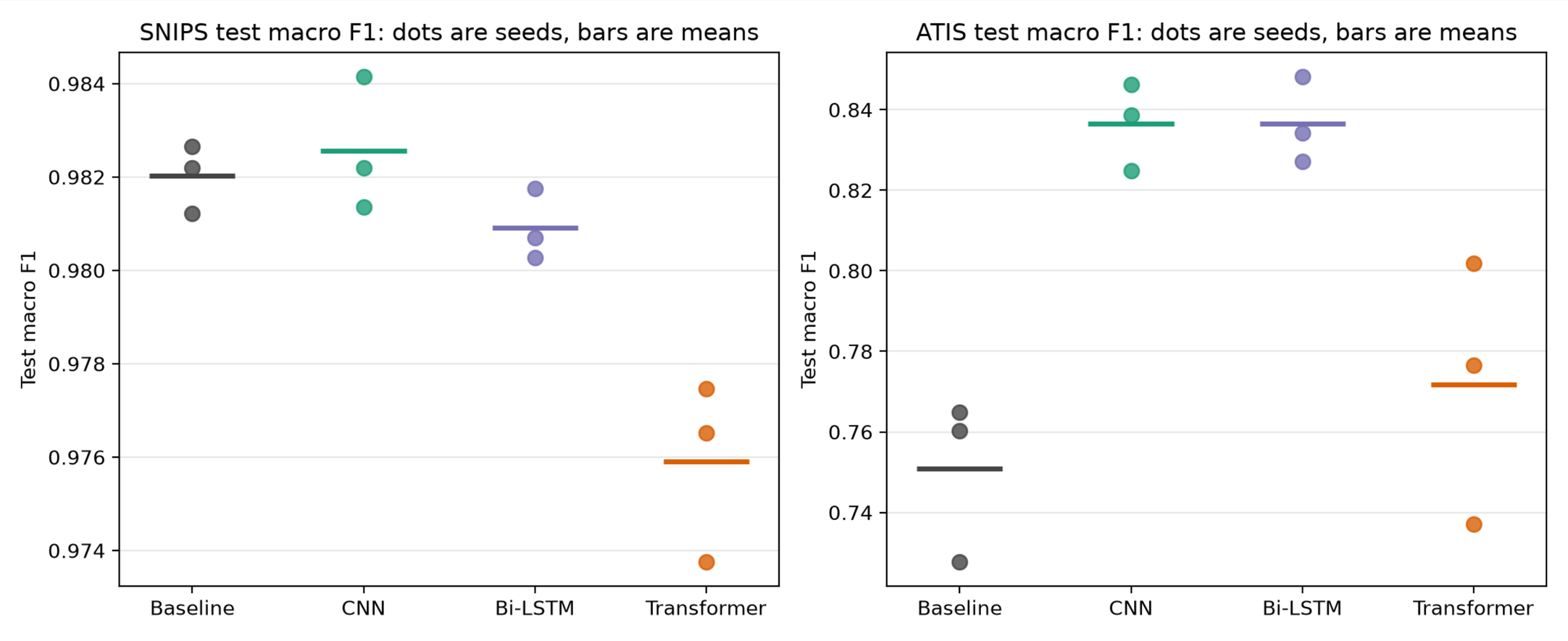
Round 3: final

Results for every seed

SNIPS	Seed 42	Seed 123	Seed 999	Mean	Spread
Baseline	0.9812	0.9822	0.9827	0.9820	0.0014
1D CNN	0.9842	0.9822	0.9814	0.9826	0.0028
Bi-LSTM	0.9807	0.9803	0.9818	0.9809	0.0015
Transformer	0.9775	0.9738	0.9765	0.9759	0.0037

ATIS	Seed 42	Seed 123	Seed 999	Mean	Spread
Baseline	0.7649	0.7602	0.7277	0.7509	0.0371
1D CNN	0.8462	0.8248	0.8387	0.8366	0.0214
Bi-LSTM	0.8343	0.8271	0.8483	0.8365	0.0212
Transformer	0.8019	0.7370	0.7765	0.7718	0.0649

The picture changed once the comparison was fair



On ATIS, the Transformer dropped from first to last among the deep models. The CNN and Bi-LSTM clearly beat the baseline.

Removing components changed little

Removed (SNIPS)	Standard	Ablated	Change	Per seed
CNN: BatchNorm	0.9826	0.9803	-0.0023	-0.0014, -0.0027, -0.0028
LSTM: backward pass	0.9809	0.9789	-0.0021	-0.0004, +0.0000, -0.0058
Transformer: positions	0.9759	0.9769	+0.0010	+0.0014, +0.0028, -0.0013

Word order adding so little fits keyword-driven intents, though we didn’t test that cause directly.

The BatchNorm comparison also crosses devices: standard CNN on CPU, ablated CNN on GPU.

What each model costs to run

Model	Parameters	ms per sample	Memory added (MB)
Baseline	25,704	0.0035	0.4
1D CNN	588,095	0.0566	7.6
Bi-LSTM	735,879	0.1118	15.4
Transformer	735,751	0.1272	17.3

The baseline runs about 16 times faster than the CNN at a comparable SNIPS score.

Intel(R) Core(TM) i9-9980HK CPU @ 2.40GHz, 6 threads, batch 64, length 35, mean of three SNIPS checkpoints

What we identified

The evidence supports

- On easy, balanced text, a simple model is enough
- On harder, imbalanced text, the CNN and LSTM win
- The Transformer showed no advantage on either dataset
- Pipeline fairness changed the ranking

We would not claim

- Statistical significance from three seeds
- That the CNN beats the LSTM on ATIS
- Tested causes for why the models differ
- Results free of device effects or earlier test exposure